

Background - Transformers Overview

Taken from [3B1B's NN playlist](#):

📺 Transformers (how LLMs work) explained visually | DL5

At initialisation, when the input sentence is given to GPT, each token is assigned a vector representation (which is a column from GPT's embeddings matrix; no. of columns = vocabulary of the model). Assume the task is to predict the next word, after the end of the sentence.

At each layer, this data matrix with the token's embeddings as its columns undergoes transformations by passing through an attention block and then an MLP. The end goal of this is to modify each token's embeddings so as to capture context from all the other tokens in the sentence. The weights of these attention and MLP blocks get tuned during the training process.

Finally, after all the layers, the model looks at the modified embedding representing the last word in the input sentence, and then generates a prediction of what the next word of the sentence will be (using softmax and, generates a probability score) since that final embedding has captured the sentence's overall context coming from the preceding words.

That final word embedding is multiplied with the 'unembedding matrix', which has one row for each word in the model's vocabulary, and number of cols = dimensionality of embeddings (basically, dimensions of unembedding matrix == that of embeddings matrix's transpose). This product gives a vector with a 'value' for each word in the vocabulary, that is then converted into a probability for that word through softmax. Inputs to softmax == "logits"

(softmax = std. Way to convert an arbitrary list of numbers into a valid probability distributions so that largest values -> 1 and smallest values -> 0); $p_i = \frac{e^{x_i}}{\sum_i (e^{x_i})}$

Temperature constant T: $\rightarrow \frac{e^{x_i}}{(\sum_i (e^{x_i}/T))}$

Larger T -> more weight to lower values -> distribution is a bit more uniform

Smaller T -> more weight to bigger values -> all weight goes to the max value.

Thus T = 0 -> always goes to most predictable next word; Higher T -> less predictable word

Attention in transformers, visually explained | DL6

Aim of attention blocks: to move token embeddings closer to what they're meant to represent w.r.t the sentence context, in vector space.

When the data (embeddings for each token) reaches the attention head, each token embedding (E_i) has specific 'query' vectors ($Q = W_q \times E_i$) that as questions pertaining to that tokens relations with other words, and vice-versa; these are obtained by multiplying certain weight matrices (W_q) with the token embeddings. This weight matrix $\Rightarrow$ params!

Example query for a noun token: whether this noun has any adjectives preceding it

A second type of matrix called key matrix (W_k , also params) is also multiplied to all E_i to get a second sequence of vectors called keys (K); these conceptually are answers to the queries. Both W_q and W_k map E_i to a smaller dimensionality!

Keys will align with i.e., be similar to queries when they closely mean the same thing i.e., the answer; eg., "fluffy blue creature" $\Rightarrow E_1 E_2 E_3$

Let Q be the query of whether E_3 is preceded by adjectives; $Q = W_q \times E_3$

Then, the keys created for E_1 and E_2 i.e., $K_1 = W_k \times E_1$ and $K_2 = W_k \times E_2$ will be closely aligned to Q in vector space!!! $\Rightarrow E_1$ and E_2 "attend" to E_3 , since their keys match E_3 's Q

To compute how well each key matches each query, we form a dot prod matrix b/w each key/query pair; Dot prod.s for appropriate matching keys and queries (since there are many different queries, and many different answers i.e., keys) will be higher since they're similar.

	a	fluffy	blue	creature	roamed	the	verdant	forest
	$\downarrow$ $\vec{E}_1$ $\downarrow W_q$ $\vec{Q}_1$	$\downarrow$ $\vec{E}_2$ $\downarrow W_q$ $\vec{Q}_2$	$\downarrow$ $\vec{E}_3$ $\downarrow W_q$ $\vec{Q}_3$	$\downarrow$ $\vec{E}_4$ $\downarrow W_q$ $\vec{Q}_4$	$\downarrow$ $\vec{E}_5$ $\downarrow W_q$ $\vec{Q}_5$	$\downarrow$ $\vec{E}_6$ $\downarrow W_q$ $\vec{Q}_6$	$\downarrow$ $\vec{E}_7$ $\downarrow W_q$ $\vec{Q}_7$	$\downarrow$ $\vec{E}_8$ $\downarrow W_q$ $\vec{Q}_8$
a $\rightarrow \vec{E}_1 \xrightarrow{W_k} \vec{K}_1$	+0.7	-83.7	-24.7	-27.8	-5.2	-89.3	-45.2	-36.1
fluffy $\rightarrow \vec{E}_2 \xrightarrow{W_k} \vec{K}_2$	-73.4	+2.9	-5.4	+93.0	-48.2	-87.3	-49.7	+7.8
blue $\rightarrow \vec{E}_3 \xrightarrow{W_k} \vec{K}_3$	-53.4	-5.7	+1.8	+93.4	-55.6	-56.0	-26.1	-62.1
creature $\rightarrow \vec{E}_4 \xrightarrow{W_k} \vec{K}_4$	-21.5	-29.7	-56.1	+4.9	-32.4	-92.3	-9.5	-28.1
roamed $\rightarrow \vec{E}_5 \xrightarrow{W_k} \vec{K}_5$	-20.1	-40.9	-87.8	-55.4	+0.6	-64.7	-96.7	-18.9
the $\rightarrow \vec{E}_6 \xrightarrow{W_k} \vec{K}_6$	-87.9	-33.3	-22.6	-31.4	+5.5	+0.6	-4.6	-96.8
verdant $\rightarrow \vec{E}_7 \xrightarrow{W_k} \vec{K}_7$	-41.2	-55.5	-42.3	-59.8	-79.0	-97.9	+3.7	+93.8
forest $\rightarrow \vec{E}_8 \xrightarrow{W_k} \vec{K}_8$	-58.9	-75.5	-91.1	-90.6	-75.6	-89.0	-70.8	+4.7

Then, we apply softmax to each column to normalize values (0 -1) $\Rightarrow$ "Attention pattern" matrix

Representing all this in mathematical form;

Attention(Q, K, V) = softmax(Q.K^T/root(d_k)) . V, where

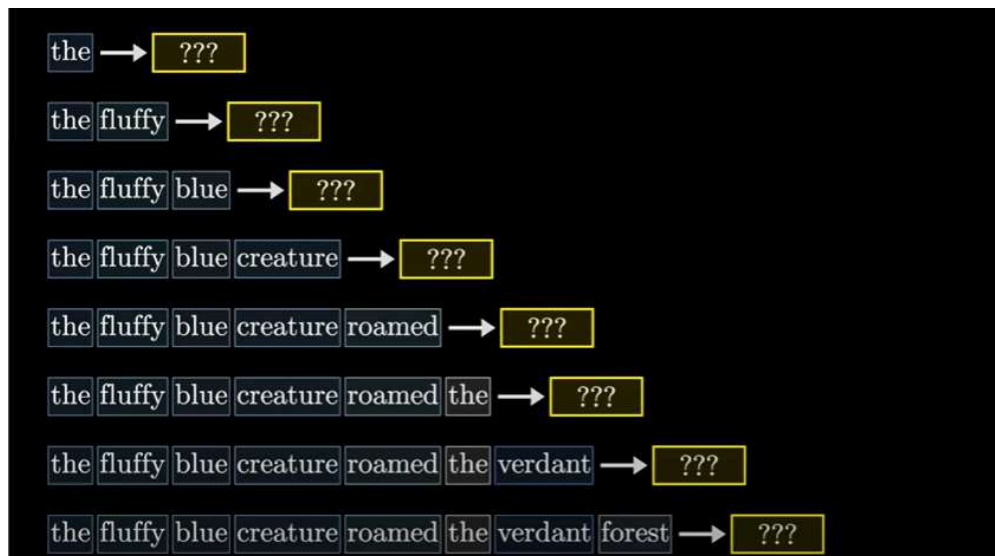
Q = [Q1 Q2 ... Qn] and K = [K1 K2 ... Kn]; dot(Q, K^T) = matrix in above img,

d_k = dimension of that key-query space; for stability; V = value vector

Training is more efficient when you make the model try to predict all possible next tokens, following every possible sub-sequence of tokens (token 1, token 1 token 2, token 1 token 2 token 3,)

And make it do this for all subsequences simultaneously.

Single training example becomes like many training examples simultaneously!



For attention, we don't want succeeding words to affect the context of the previous words (since it's next-work predicting based on prev text alone); so the later tokens must be masked somehow. ⇒ Masking process!

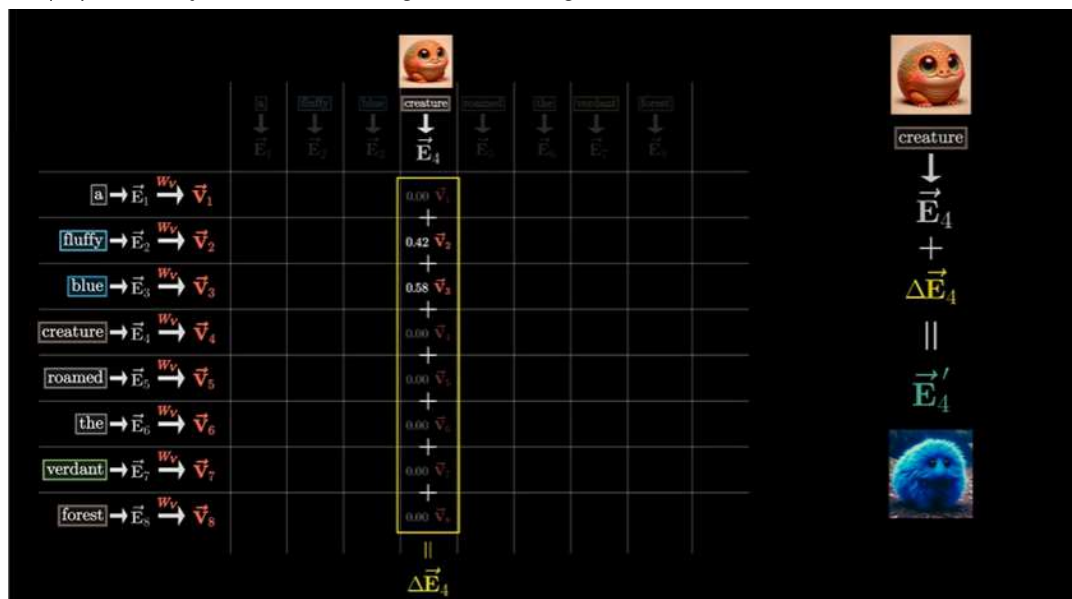
To do this, when considering one token, the key-query dot prods coming from all future tokens is set to -infinity, so that when we do softmax it becomes 0 and col.s are still normalized.

Also, size(attention pattern) == square of size(context window) i.e., input length ⇒ scales quadratically! Biggest problem when increasing context window size.

Now see our example of “blue fluffy creature”; we want the embedding of creature to move towards something that's more representative of a blue fluffy creature in vector space. For this, we use a “value matrix” which is multiplied with both E1 and E2 (blue and fluffy) and these products are added to E3 (for creature) in the hopes that it will nudge E3 towards something more representative. Basically, when a value matrix (Wv) is multiplied with each embedding, it gives value vectors V (V1, V2, ... one for each embedding/token); V_i = Wv x E_i. These represent how significant of a change in a specific token's embedding is caused by other specific tokens

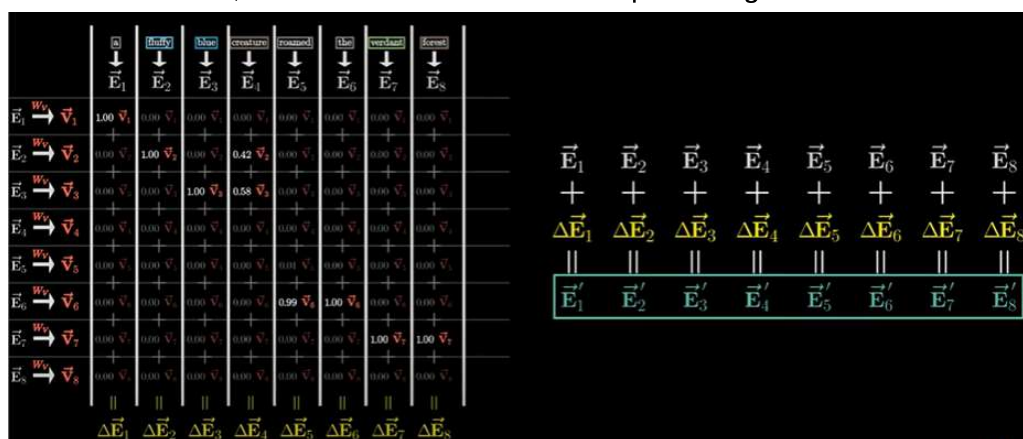
Then, do a dot prod with V and each column i.e., V . (col) from the attention pattern matrix; high dot. Prods b/w Q and K will let the respective token's V_i to still be there whereas the tokens that

do not influence the context of the embedding in consideration, have their value V_i become zero since the dot prod is very low. Then, you sum up all the values in this col to get a number, $\text{del}(E_i)$, which you add to the og. embedding E_i



E_4 now represents a “fluffy creature” more than just a “creature”

Do this for each embedding, and now the final embeddings (E') after adding all the contributions from other tokens, have more context from their preceding tokens.



This whole process culminating in this addition $\Rightarrow$ “One head of self- attention”

Dimensions: no. of cols in Q ($Q_1 Q_2 \dots Q_N$) = no. in $K = n$ = embedding dimensionality

No. of rows in both $\Rightarrow$ specific to that key query space; can be smaller $\Rightarrow$ dimension reduction?

For efficiency, number of params in W_v = no. in W_q + no. in W_k

Multi-headed attention $\Rightarrow$ do these operations in parallel; each operation has distinct ke, query and value maps. $\Rightarrow$ gives the model the ability to learn many distinct ways that context changes meaning.

Biggest success of attention $\Rightarrow$ highly parallelizable

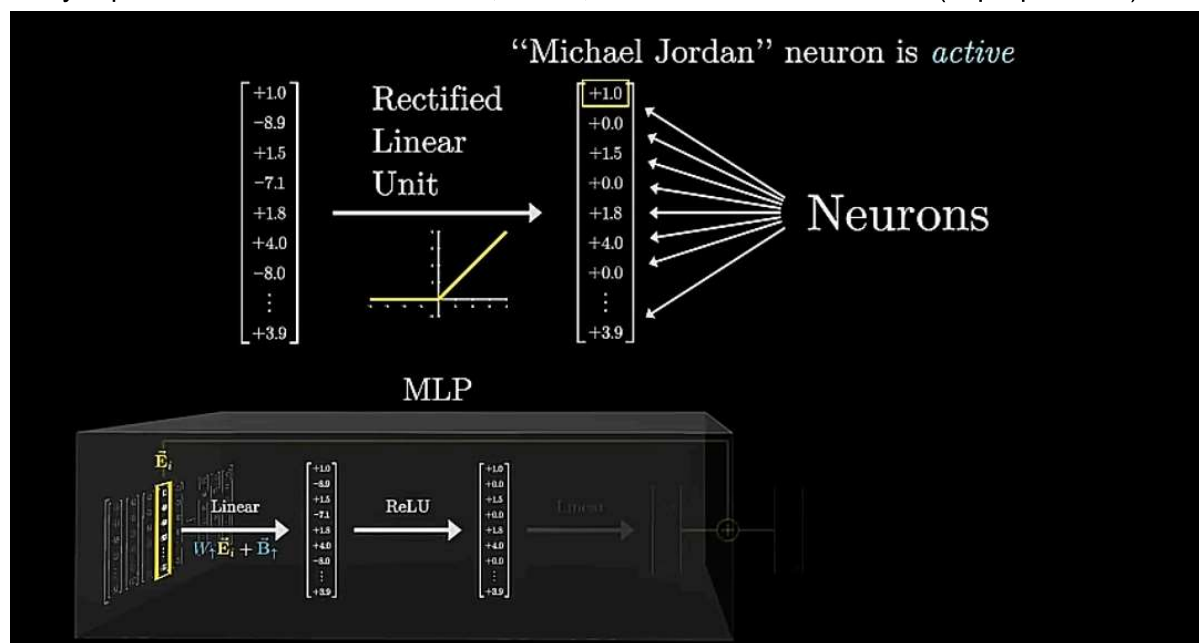
▶ How might LLMs store facts | DL7

Facts seem to be stored in the MLP blocks! $\Rightarrow$ nearly 2/3 rds of GPT-3's params

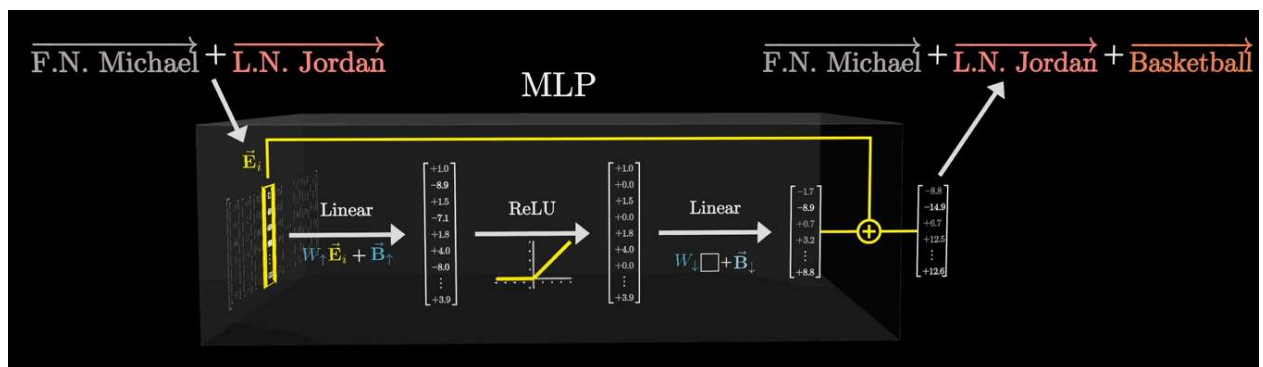
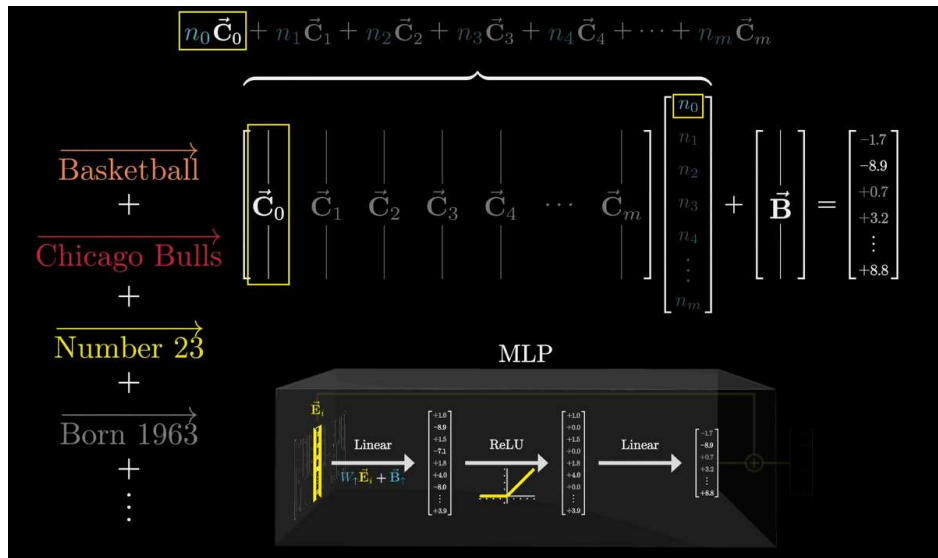
In an MLP, each token embedding (final output from attention block) goes through the MLP independently, through linear transformations, activation functions and (standard FFNN stuff) and this modified embedding gets added to the original input token embedding, and this sum is then sent as input to the next layer's attention block.

In the first linear transformation i.e., matrix multiplication with the embedding, again this matrix is a param whose rows can be thought of as probing for certain features regarding the embedding being processed, asking for true (1) or false (0). Bias vectors are added to this product; it's also made of params learned from data.

This goes through an activation function; note that each element of the vector now is referred to as a 'neuron'; if these neurons are 0 then the concept represented by a column in the next weight matrix will have no influence on the token's embedding.. thing is, single neurons don't really represent features on their own, rather, a combination of them do (superposition!)



This is then multiplied with another weight matrix, this time whose no. of rows = embedding dimensionality; so each column can be thought of representing one direction i.e., concept in vector space.



"Micheal jordan" embedding now has the basketball context in it!!!

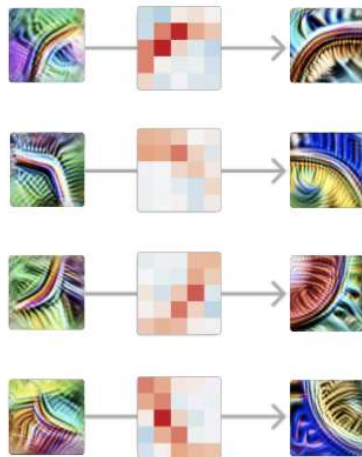
Circuits

Zoom In: An Introduction to Circuits

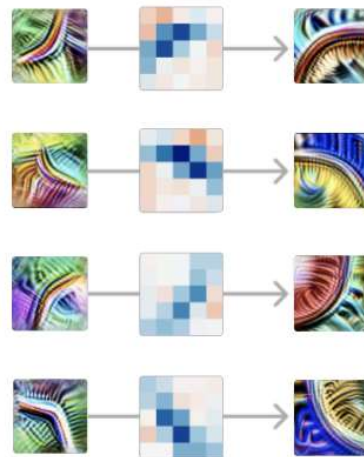
“All neurons in our network are formed from linear combinations of neurons in the previous layer, followed by ReLU. If we can understand the features in both layers, shouldn't we also be able to understand the connections between them? To explore this, we find it helpful to study **circuits: sub-graphs of the network, consisting of a set of tightly linked features and the weights between them.**”



Curve detectors are **excited** by earlier detectors in **similar orientations**...



... and **inhibited** by earlier detectors in **opposing orientations**.



In `mixed4c`, a mid-late layer of InceptionV1, there is a car detecting neuron. Using features from the previous layers, it looks for wheels at the bottom of its convolutional window, and windows at the top.

Windows (4b:237) excite the car detector at the top and inhibit at the bottom.

Car Body (4b:491) excites the car detector, especially at the bottom.

Wheels (4b:373) excite the car detector at the bottom and inhibit at the top.

